# Claude Code + Codex 本地开发环境完整部署教程

> 面向读者：完全没有开发环境的新手；一台全新的 Windows 11 电脑；尚未安装 VS Code、Git、Node.js、Python、Claude Code、OpenAI Codex CLI。
>
> 文档目标：从零完成 Windows 11 本地 AI 编程环境部署，最终具备 AI 辅助编程、终端运行、Git 项目管理、API 调用、本地项目开发能力。
>
> 官方来源原则：所有软件下载均使用官方网站或官方文档入口，不使用第三方下载站。
>
> 编写日期：2026-05-12。

## 目录


- 第 1 章 Windows 基础准备
- 第 2 章 安装 VS Code
- 第 3 章 安装 Git
- 第 4 章 安装 Node.js
- 第 5 章 安装 Python
- 第 6 章 安装 Claude Code
- 第 7 章 安装 OpenAI Codex CLI
- 第 8 章 配置 VS Code AI 开发环境
- 第 9 章 配置终端环境
- 第 10 章 API Key 配置
- 第 11 章 第一个 AI 项目实战
- 第 12 章 常见报错大全


## 官方链接总览

| 工具 | 官方网站 | 官方下载或文档 | 推荐选择 |
|---|---|---|---|
| Windows Terminal | https://learn.microsoft.com/windows/terminal/ | https://learn.microsoft.com/windows/terminal/install | Windows 11 通常已内置；没有则从 Microsoft Store 或官方 GitHub 安装 |
| VS Code | https://code.visualstudio.com/ | https://code.visualstudio.com/download | Windows User Installer x64 Stable |
| Git | https://git-scm.com/ | https://git-scm.com/downloads/win | Git for Windows 64-bit 最新稳定版 |
| Node.js | https://nodejs.org/ | https://nodejs.org/en/download/ | LTS 版本；截至本文核对时官网显示 v24.15.0 LTS |
| Python | https://www.python.org/ | https://www.python.org/downloads/windows/ | 最新稳定版 64-bit Windows installer |

| 工具 | 官方网站 | 官方下载或文档 | 推荐选择 |
| --- | --- | --- | --- |
| Claude Code | https://claude.ai/code | https://docs.anthropic.com/en/docs/claude-code/getting-started | 官方 npm 安装或 Windows PowerShell 官方安装脚本 |
| OpenAI Codex CLI | https://github.com/openai/codex | https://help.openai.com/en/articles/11096431-openai-codex-cli-getting-started | `npm install -g @openai/codex` |
| OpenAI API | https://platform.openai.com/ | https://platform.openai.com/docs/ | 创建 API Key 并配置 `OPENAI_API_KEY` |
| Anthropic API | https://console.anthropic.com/ | https://docs.anthropic.com/ | 创建 API Key 并配置 `ANTHROPIC_API_KEY` |

> ⚠ **Warning**
> 技术工具版本会持续更新。本文给出的“推荐版本”以 2026-05-12 可查官方信息为准。实际安装时，如果官网显示更新的 LTS 或稳定版本，优先选择官网当前推荐的稳定版。

---

# 第 1 章 Windows 基础准备

本章目标是让一台全新的 Windows 11 电脑具备“适合开发”的基础状态。不要跳过本章。很多后续错误，例如 `node 不是内部或外部命令`、`git 不是内部或外部命令`、无法显示 `.py` 文件扩展名、找不到项目目录，本质上都来自基础设置不清楚。

## 1.1 检查 Windows 版本

### 步骤

确认当前电脑是 Windows 11，并确认系统架构是 64 位。

### 原理

VS Code、Git、Node.js、Python、Claude Code、Codex CLI 都支持 Windows，但本教程面向 Windows 11。系统架构决定你应该下载 x64、Arm64 还是其他版本。大多数普通笔记本和台式机是 x64。如果下载错架构，安装器可能无法运行，或者安装后命令不可用。

### 操作

方法一：使用图形界面。

1. 按 `Win + I` 打开“设置”。
2. 点击“系统”。
3. 点击“关于”。
4. 查看“Windows 规格”和“设备规格”。
5. 重点确认：
    - 版本：Windows 11。

- 系统类型：64 位操作系统，基于 x64 的处理器；或 ARM64。

方法二：使用 PowerShell。

1. 右键点击开始菜单。
2. 选择“终端”或“Windows PowerShell”。
3. 输入：

```
winver
```

再输入：

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion, OsArchitecture
```

## 验证

预期看到类似结果：

```
WindowsProductName Windows 11 Pro
WindowsVersion     24H2
OsArchitecture     64-bit
```

只要确认是 Windows 11 且系统架构清楚，就可以继续。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `Get-ComputerInfo` 输出很多内容，看不懂 | 命令返回了完整系统信息 | 使用本文提供的 `Select-Object` 版本，只看三个字段 |
| 电脑是 Windows 10 | 仍可安装大部分工具，但本文截图和路径以 Windows 11 为准 | 建议升级到 Windows 11 或严格按工具官网的 Windows 10 说明操作 |
| 系统是 ARM64 | 安装包不能盲目选择 x64 | VS Code、Node.js 等优先选择 Arm64 下载项；没有 Arm64 时再查官方兼容说明 |

# 1.2 理解 PowerShell、CMD 和管理员权限

## 步骤

学习如何打开普通 PowerShell、管理员 PowerShell，以及什么时候需要管理员权限。

## 原理

Windows 下常见终端有 CMD、PowerShell、Git Bash 和 Windows Terminal。终端是你输入命令的地方。管理员权限相当于“系统级修改权限”，例如安装到 `C:\Program Files`、修改系统环境变量、安装某些系统组件时可能需要。日常开发不建议一直使用管理员终端，因为误操作风险更高。

## 操作

打开普通 PowerShell：

1. 右键点击开始菜单。
2. 点击“终端”或“Windows PowerShell”。
3. 如果标题栏没有“管理员”，说明是普通终端。

打开管理员 PowerShell：

1. 点击开始菜单。
2. 搜索 `PowerShell`。
3. 右键“Windows PowerShell”。
4. 选择“以管理员身份运行”。
5. 弹出 UAC 窗口时点击“是”。

查看当前是否为管理员：

```
([Security.Principal.WindowsPrincipal]
[Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.Windows
BuiltInRole]::Administrator)
```

## 验证

如果输出：

```
True
```

说明当前是管理员 PowerShell。

如果输出：

```
False
```

说明当前是普通 PowerShell。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 找不到 PowerShell | Windows Terminal 可能已经替代入口 | 打开“终端”，默认通常就是 PowerShell |
| 弹窗要求管理员密码 | 当前账户不是管理员 | 联系电脑管理员，或改用 User Installer 类型安装工具 |
| 安装软件总失败 | 可能没有权限写入系统目录 | 优先选择 VS Code User Installer、Node.js 当前用户安装，必要时再使用管理员权限 |

> **Tip**
> 新手原则：能用普通权限完成的安装，不要使用管理员权限；必须修改系统级设置时，再临时打开管理员终端。

## 1.3 配置 PowerShell 执行策略

### 步骤

设置当前用户的 PowerShell 脚本执行策略，避免后续安装工具或运行脚本时被默认策略拦截。

### 原理

PowerShell 的执行策略用于限制 `.ps1` 脚本运行。全新 Windows 电脑可能禁止运行本地脚本。很多开发工具会生成 PowerShell 启动脚本，例如 npm 全局命令可能同时生成 `.ps1` 文件。如果策略过严，可能出现“无法加载文件，因为在此系统上禁止运行脚本”。

### 操作

打开普通 PowerShell，执行：

```
Get-ExecutionPolicy -List
```

如果 `CurrentUser` 是 `Undefined` 或 `Restricted`，可以设置为 `RemoteSigned`：

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

出现确认提示时输入：

```
Y
```

### 验证

再次运行：

```
Get-ExecutionPolicy -List
```

预期看到：

```
CurrentUser RemoteSigned
```

### 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 提示权限不足 | 可能尝试修改 LocalMachine 策略 | 使用 `-Scope CurrentUser`，不要改系统级策略 |
| 公司电脑不允许修改 | 组策略限制 | 联系管理员，或使用 CMD/Git Bash 运行对应命令 |

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 仍提示脚本被禁止 | 终端未重启或策略被覆盖 | 关闭所有终端窗口，重新打开 PowerShell |

> ⚠ **Warning**
> 不建议为了省事执行 `Set-ExecutionPolicy Unrestricted`。`RemoteSigned` 对新手更合适：本地脚本可运行，来自网络的脚本需要签名或解除阻止。

# 1.4 理解系统环境变量和 PATH

## 步骤

理解环境变量是什么，特别是 `PATH` 的作用。

## 原理

环境变量是操作系统保存的一组“全局配置”。例如：

| 环境变量 | 作用 |
| --- | --- |
| `PATH` | 告诉系统去哪些文件夹寻找可执行命令 |
| `OPENAI_API_KEY` | 保存 OpenAI API Key |
| `ANTHROPIC_API_KEY` | 保存 Anthropic API Key |
| `USERPROFILE` | 当前用户目录，例如 `C:\Users\Lewis` |

当你在终端输入：

```
node -v
```

系统会在 `PATH` 里的每个目录寻找 `node.exe`。如果 Node.js 已安装，但安装目录不在 `PATH`，系统就会提示 `node` 不是内部或外部命令。

## 操作

查看当前用户的 `PATH`：

```
$env:Path -split ';'
```

打开图形界面设置环境变量：

1. 按 `Win + S` 搜索“环境变量”。
2. 点击“编辑系统环境变量”。
3. 点击“环境变量”。
4. 上半部分是“用户变量”，只影响当前用户。
5. 下半部分是“系统变量”，影响所有用户，通常需要管理员权限。

## 验证

执行：

```
where.exe powershell
```

预期输出某个 `powershell.exe` 路径，说明系统能通过 `PATH` 找到命令。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 修改 PATH 后命令仍不可用 | 旧终端没有读取新环境变量 | 关闭所有终端，重新打开 |
| PATH 中有重复项 | 多次安装或手动添加导致 | 不急于删除；确认工具可用后再整理 |
| 把文件路径加入 PATH | PATH 应该添加“文件夹路径” | 添加 `C:\Program Files\nodejs\`，不要添加 `node.exe` 本身 |

# 1.5 显示文件扩展名

## 步骤

让 Windows 文件资源管理器显示 `.py`、`.js`、`.md`、`.json` 等扩展名。

## 原理

扩展名表示文件类型。新手常见问题是创建了 `app.py.txt`，自己以为是 Python 文件，但系统实际认为它是文本文件。显示扩展名可以避免后续项目文件命名错误。

## 操作

1. 打开任意文件夹。
2. 点击顶部“查看”。
3. 点击“显示”。
4. 勾选“文件扩展名”。
5. 建议同时勾选“隐藏的项目”，方便查看 `.git`、`.env` 等开发文件。

## 验证

新建一个文本文件，如果显示为：

```
新建 文本文档.txt
```

说明扩展名显示成功。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 文件仍不显示 `.txt` | 没有勾选文件扩展名 | 重新检查“查看 -> 显示 -> 文件扩展名” |
| 改名时提示可能不可用 | Windows 提醒你正在修改扩展名 | 如果你确实要把 `.txt` 改成 `.py`，点击确认 |
| `.git` 文件夹看不到 | 隐藏项目未显示 | 勾选“隐藏的项目” |

# 1.6 建立开发目录规范

## 步骤

创建统一的开发目录，例如：

```
C:\Dev
```

并约定以后所有项目都放在这个目录下。

## 原理

开发项目最好不要放在桌面、下载目录、微信/QQ 文件目录、OneDrive 自动同步目录中。原因是：

1. 路径太长或包含中文、空格、特殊符号时，部分工具可能出错。
2. OneDrive 同步可能锁定文件，导致 npm、pip、Git 操作失败。
3. 桌面文件太杂，不利于长期维护。
4. AI 编程工具会读取项目目录，清晰的目录结构可以减少误操作。

## 操作

打开 PowerShell，执行：

```
New-Item -ItemType Directory -Force -Path C:\Dev
New-Item -ItemType Directory -Force -Path C:\Dev\tools
New-Item -ItemType Directory -Force -Path C:\Dev\projects
New-Item -ItemType Directory -Force -Path C:\Dev\playground
```

建议用途：

| 目录 | 用途 |
| --- | --- |
| `C:\Dev\tools` | 放手动下载的工具、临时安装包 |
| `C:\Dev\projects` | 放正式项目 |
| `C:\Dev\playground` | 放学习、测试、临时代码 |

## 验证

执行：

```
Get-ChildItem C:\Dev
```

预期看到：

```
tools
projects
playground
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 提示拒绝访问 | 当前账户没有 C 盘根目录写入权限 | 改用 `C:\Users\<你的用户名>\Dev` |
| 目录已存在 | `-Force` 会复用已有目录 | 属于正常情况 |
| 路径中有中文是否可以 | 大多数工具支持中文，但排错成本更高 | 新手建议使用英文路径 |

---

# 第 2 章 安装 VS Code

VS Code 是本教程的开发核心。它不是唯一的编辑器，但它对新手最友好：编辑代码、打开终端、管理 Git、安装 AI 插件、调试程序，都可以在一个窗口里完成。

## 2.1 为什么 VS Code 是 AI 开发核心

### 步骤

理解 VS Code 在 AI 编程环境中的位置。

### 原理

AI 编程不是只和聊天窗口对话。真实开发需要同时处理：

| 能力 | VS Code 中对应功能 |
| --- | --- |
| 写代码 | 编辑器、语法高亮、自动补全 |
| 运行代码 | 集成终端 |
| 管理项目 | 文件资源管理器 |
| 管理版本 | Git 源代码管理面板 |
| 调试错误 | 调试器、断点、变量查看 |
| 调用 AI | Claude Code、Copilot、Continue、Cline、Roo Code 等插件 |

| 能力 | VS Code 中对应功能 |
| --- | --- |
| 阅读文档 | Markdown 预览 |

VS Code 的价值在于“把项目上下文集中在一个地方”。AI 工具越依赖上下文，越需要一个结构清晰的工作区。

## 操作

本章安装 VS Code，并进行基础设置。后续第 8 章会专门配置 AI 插件。

## 验证

安装完成后，能够在命令行执行：

```
code --version
```

并能使用：

```
code C:\Dev\projects
```

打开项目目录。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 能打开 VS Code，但 `code` 命令不可用 | 安装时没有加入 PATH，或终端未重启 | 重新安装并勾选 Add to PATH，或重启终端 |
| VS Code 打开很慢 | 插件过多或电脑性能不足 | 先只安装必要插件 |
| AI 插件互相弹窗 | 多个插件同时接管补全 | 第 8 章会给出新手推荐组合 |

# 2.2 下载 VS Code

## 步骤

从 VS Code 官方网站下载 Windows 安装器。

## 原理

使用官方网站可以避免第三方下载站捆绑广告软件、旧版本安装包或被篡改的安装器。VS Code 有 User Installer 和 System Installer 两种常见安装方式。

| 安装器 | 特点 | 推荐人群 |
| --- | --- | --- |
| User Installer | 安装到当前用户目录，不需要管理员权限 | 新手、个人电脑、学校电脑 |
| System Installer | 安装到 `Program Files`，影响所有用户，需要管理员权限 | 多用户电脑、实验室统一部署 |

| 安装器 | 特点 | 推荐人群 |
| --- | --- | --- |
| ZIP | 免安装压缩包 | 高级用户、临时环境 |

新手推荐 `User Installer x64`。

## 操作

1. 打开浏览器。
2. 访问官方网站：https://code.visualstudio.com/download
3. 找到 Windows 区域。
4. 选择 `User Installer`。
5. 如果你的电脑是常见 Intel/AMD 处理器，选择 `x64`。
6. 下载完成后，文件名通常类似：

```
VSCodeUserSetup-x64-*.exe
```

## 验证

在“下载”文件夹中能看到 VS Code 安装器，并且发布者是 Microsoft。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 不知道选 x64 还是 Arm64 | 未确认系统架构 | 回到第 1 章查看系统类型 |
| 浏览器提示不安全 | 下载 `.exe` 时浏览器常规提醒 | 确认网址是 `code.visualstudio.com` |
| 下载很慢 | 网络问题 | 稍后重试，或使用 Microsoft Store 官方入口 |

# 2.3 安装 VS Code

## 步骤

运行 VS Code 安装器，并选择适合开发的安装选项。

## 原理

安装过程中的几个勾选项会影响日常开发效率。特别是“添加到 PATH”和“右键菜单打开”，会让你能从终端或文件夹快速进入 VS Code。

## 操作

双击安装器，按以下建议选择：

1. 接受许可协议。
2. 安装位置保持默认。

3. 开始菜单文件夹保持默认。
4. “选择其他任务”页面建议勾选：
    - Add "Open with Code" action to Windows Explorer file context menu。
    - Add "Open with Code" action to Windows Explorer directory context menu。
    - Register Code as an editor for supported file types。
    - Add to PATH。
5. 点击安装。
6. 安装完成后勾选“Launch Visual Studio Code”。

## 验证

关闭所有 PowerShell 窗口，重新打开 PowerShell，执行：

```
code --version
```

预期输出版本号，例如：

```
1.xxx.x
```

再执行：

```
code C:\Dev\projects
```

预期 VS Code 打开 `C:\Dev\projects` 文件夹。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `code` 不是内部或外部命令 | 未勾选 Add to PATH 或终端未重启 | 重启终端；仍失败则重新安装并勾选 Add to PATH |
| 右键菜单没有 Open with Code | 安装时未勾选右键菜单 | 重新运行安装器，选择 Modify 或重新安装 |
| 安装需要管理员权限 | 可能下载了 System Installer | 回官网下载 User Installer |

# 2.4 设置中文界面

## 步骤

安装 VS Code 中文语言包。

## 原理

新手刚开始更适合使用中文界面理解菜单。但长期看，建议逐步熟悉英文术语，因为报错、文档、插件配置大多是英文。

## 操作

1. 打开 VS Code。
2. 点击左侧扩展图标，或按：

```
Ctrl + Shift + X
```

3. 搜索：

```
Chinese (Simplified) Language Pack
```

4. 找到 Microsoft 发布的中文简体语言包。
5. 点击 Install。
6. 安装后点击 Restart。

## 验证

VS Code 菜单变为中文，例如“文件”“编辑”“终端”。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 搜不到插件 | 网络或 Marketplace 连接问题 | 稍后重试；检查代理或网络 |
| 安装后仍是英文 | 未重启 VS Code | 按提示 Restart |
| 想切回英文 | 语言配置为中文 | 按 `Ctrl+Shift+P`，搜索 `Configure Display Language`，选择 `en` |

# 2.5 推荐字体、主题与编辑器设置

## 步骤

配置适合长期编程的字体、主题、自动保存和基础编辑体验。

## 原理

开发环境要长期使用。字体影响可读性，主题影响疲劳程度，自动保存可以减少新手忘记保存导致的运行结果不一致。

## 操作

打开设置：

```
Ctrl + ,
```

推荐设置：

| 设置项 | 推荐值 | 原因 |
| --- | --- | --- |
| Font Family | `Cascadia Code, Consolas, monospace` | Windows 上清晰稳定 |
| Font Size | 14 或 15 | 新手阅读更轻松 |
| Tab Size | 4 | Python 友好 |
| Word Wrap | on | 长行自动换行 |
| Auto Save | afterDelay | 防止忘记保存 |
| Format On Save | 先关闭 | 新手先避免自动格式化造成困惑 |
| Minimap | 可关闭 | 小屏幕减少干扰 |

也可以直接打开 `settings.json` 添加：

```json
{
  "editor.fontFamily": "Cascadia Code, Consolas, monospace",
  "editor.fontSize": 14,
  "editor.tabSize": 4,
  "editor.wordWrap": "on",
  "files.autoSave": "afterDelay",
  "files.autoSaveDelay": 1000,
  "terminal.integrated.defaultProfile.windows": "PowerShell"
}
```

主题推荐：

| 主题 | 场景 |
| --- | --- |
| Default Dark Modern | 默认深色，稳定 |
| Default Light Modern | 白天、打印截图更清楚 |
| GitHub Theme | 接近 GitHub 阅读体验 |
| One Dark Pro | 流行深色主题 |

## 验证

新建文件 `test.py`，输入几行代码，确认字体清晰、自动保存生效。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 中文显示不清楚 | 字体不包含中文优化 | 使用默认 UI 字体；代码字体只影响编辑器 |
| 保存后格式变了 | 开启了 Format On Save | 暂时关闭，等熟悉格式化工具后再开启 |
| 终端字体太小 | 终端有独立字号 | 设置 `terminal.integrated.fontSize` |

# 2.6 配置 VS Code 终端

## 步骤

把 VS Code 默认终端设置为 PowerShell。

## 原理

VS Code 集成终端让你不用离开编辑器就能运行命令。新手在 Windows 上先统一使用 PowerShell，可以减少“这个命令到底在哪个终端运行”的混乱。Git Bash 和 CMD 后续也会介绍，但默认先选 PowerShell。

## 操作

方法一：图形界面。

1. 打开 VS Code。
2. 按 `Ctrl + Shift + P`。
3. 输入：

```
Terminal: Select Default Profile
```

4. 选择 `PowerShell`。
5. 打开新终端：

```
Ctrl + `
```

方法二：`settings.json`。

```
{
  "terminal.integrated.defaultProfile.windows": "PowerShell"
}
```

## 验证

打开 VS Code 终端，输入：

```
$PSVersionTable.PSVersion
```

预期输出 PowerShell 版本号。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 默认打开 CMD | 默认终端配置不是 PowerShell | 重新执行 Select Default Profile |
| 终端打不开 | VS Code 终端进程异常 | 重启 VS Code |
| 命令在外部 PowerShell 可用，在 VS Code 不可用 | VS Code 启动时读取的是旧环境变量 | 完全退出 VS Code 后重新打开 |

---

# 第 3 章 安装 Git

Git 是现代开发的基础设施。即使你暂时不打算上传代码到 GitHub，也应该安装 Git，因为 AI 编程工具在修改文件前后需要版本管理帮助你比较、回退和审查变化。

## 3.1 Git 的作用

### 步骤

理解 Git 在开发流程中的位置。

### 原理

Git 是版本控制工具。它能记录项目每一次重要变化，让你知道：

1. 哪些文件改了。
2. 每一行代码是谁改的。
3. 某次修改引入了什么问题。
4. 如何回到之前可运行的版本。

AI 辅助编程时，Git 更重要。因为 AI 可以一次修改多个文件，Git 能帮你审查差异，避免“改坏了不知道哪里坏”。

### 操作

本章会安装 Git for Windows，并完成基础身份配置。

### 验证

安装完成后执行：

```
git --version
```

能输出 Git 版本号。

### 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 不理解 Git 是否必须 | 本地项目也需要版本管理 | 即使不上传 GitHub，也建议使用 Git |
| 担心命令复杂 | 初期只需掌握 `init/status/add/commit` | 后续项目实战会演示 |
| AI 工具提示建议使用 Git | 当前目录不是 Git 仓库 | 执行 `git init` |

## 3.2 下载 Git for Windows

### 步骤

从 Git 官方网站下载 Windows 安装器。

### 原理

Windows 不自带 Git。Git for Windows 提供 `git.exe`、Git Bash、OpenSSH 等工具。Claude Code 官方说明 Windows 可通过 WSL 或 Git for Windows 使用；因此安装 Git for Windows 对 Claude Code 也有帮助。

### 操作

1. 打开官网：https://git-scm.com/downloads/win
2. 下载 `64-bit Git for Windows Setup`。
3. 如果自动跳转到 Git for Windows 项目页面，确认来源仍是官方入口。
4. 保存安装器。

### 验证

下载文件通常类似：

```
Git-*-64-bit.exe
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 页面自动下载 | Git 官网正常行为 | 等待下载完成即可 |
| 不知道 portable 还是 setup | portable 是便携版 | 新手选择 Setup 安装器 |
| 公司网络拦截 | 下载 `.exe` 被限制 | 使用管理员允许的官方下载方式 |

# 3.3 安装 Git

## 步骤

按推荐选项安装 Git for Windows。

## 原理

Git 安装器选项很多。新手不要随意改高级选项，使用推荐值即可。最关键的是让 Git 能在 PowerShell 中使用，这样 VS Code 终端也能运行 Git。

## 操作

双击安装器，建议如下：

| 安装页面 | 推荐选项 | 说明 |
| --- | --- | --- |
| Select Destination Location | 默认 | 安装到官方默认目录 |
| Select Components | 默认；可勾选 Git Bash Here | 右键菜单更方便 |
| Choosing the default editor | Visual Studio Code | Git 提交信息可用 VS Code 编辑 |
| Initial branch name | Override，填 `main` | 现代项目常用 `main` |
| PATH environment | Git from the command line and also from 3rd-party software | 让 PowerShell、VS Code 都能用 Git |
| SSH executable | Use bundled OpenSSH | 使用 Git 自带 SSH |
| HTTPS transport backend | Use OpenSSL library | 默认即可 |
| Line endings | Checkout Windows-style, commit Unix-style | Windows 推荐默认值 |
| Terminal emulator | Use MinTTY | Git Bash 使用体验更好 |
| Default behavior of `git pull` | Default | 新手先保持默认 |
| Credential helper | Git Credential Manager | 保存 GitHub 登录凭据 |

| 安装页面 | 推荐选项 | 说明 |
| --- | --- | --- |
| Extra options | Enable file system caching | 提升性能 |
| Experimental options | 不勾选 | 避免不稳定功能 |

## 验证

关闭所有终端，重新打开 PowerShell：

```
git --version
```

预期：

```
git version 2.x.x.windows.x
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `git` 不是内部或外部命令 | PATH 没配置或终端未重启 | 重启终端；仍失败则重新安装并选择 command line 选项 |
| 安装选错默认编辑器 | 之后仍可修改 | 执行 `git config --global core.editor "code --wait"` |
| 不知道 Git Bash 是什么 | Git 自带的类 Linux 终端 | 日常可用 PowerShell，遇到 Bash 命令再用 Git Bash |

# 3.4 配置 Git 用户身份

## 步骤

设置 Git 提交时使用的用户名和邮箱。

## 原理

Git 每次提交都需要记录作者信息。用户名和邮箱不是 Windows 账户，而是 Git 提交记录里的身份。以后上传 GitHub 时，邮箱最好与 GitHub 账号邮箱一致。

## 操作

在 PowerShell 执行：

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

建议把 `Your Name` 换成英文名或常用 ID，把邮箱换成你的 GitHub 邮箱。

设置默认分支名：

```
git config --global init.defaultBranch main
```

设置 VS Code 为默认编辑器：

```
git config --global core.editor "code --wait"
```

查看配置：

```
git config --global --list
```

## 验证

预期输出包含：

```
user.name=Your Name
user.email=your_email@example.com
init.defaultbranch=main
core.editor=code --wait
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 邮箱写错了 | Git 配置可以覆盖 | 重新执行 `git config --global user.email "正确邮箱"` |
| `code --wait` 不可用 | VS Code PATH 未配置 | 回到第 2 章修复 `code` 命令 |
| 中文名乱码 | 终端编码或显示问题 | 用户名可用英文，提交信息可后续再配置编码 |

# 3.5 Git Bash 与 PowerShell 区别

## 步骤

了解何时使用 PowerShell，何时使用 Git Bash。

## 原理

两者都能运行命令，但语法不同。

| 对比项 | PowerShell | Git Bash |
| --- | --- | --- |
| 来源 | Windows 现代 Shell | Git for Windows 附带 |
| 路径风格 | `C:\Dev\projects` | `/c/Dev/projects` |
| 常见命令 | `Get-ChildItem`、`Set-Item` | `ls`、`export` |

| 对比项 | PowerShell | Git Bash |
| --- | --- | --- |
| 适合场景 | Windows 管理、环境变量、VS Code 默认终端 | 运行 Linux 风格命令、部分开源工具说明 |
| 新手推荐 | 默认使用 | 遇到 Bash 文档再使用 |

### 操作

在 PowerShell 查看目录：

```
Get-ChildItem C:\Dev
```

在 Git Bash 查看目录：

```
ls /c/Dev
```

### 验证

两个终端都能看到同一个 `C:\Dev` 目录下的内容。

### 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| Git Bash 中 `C:\Dev` 不工作 | Bash 使用类 Unix 路径 | 改成 `/c/Dev` |
| PowerShell 中 `export` 不工作 | `export` 是 Bash 命令 | PowerShell 用 `$env:变量名="值"` |
| 文档命令复制后报错 | 文档针对不同 Shell | 先确认代码块标注是 PowerShell、CMD 还是 Bash |

---

# 第 4 章 安装 Node.js

Node.js 是 JavaScript 的运行环境，npm 是 Node.js 附带的包管理器。Claude Code 和 OpenAI Codex CLI 的推荐安装方式都可以通过 npm 完成，因此 Node.js 是本教程的关键依赖。

## 4.1 Node.js 的作用

### 步骤

理解 Node.js、npm、全局命令之间的关系。

## 原理

浏览器可以运行 JavaScript，但很多开发工具需要在本地电脑运行 JavaScript，这就需要 Node.js。npm 是 Node.js 生态的包管理器，可以下载安装到本机的命令行工具。

关系如下：

| 名称 | 作用 | 类比 |
| --- | --- | --- |
| Node.js | 运行 JavaScript 程序 | Python 解释器 |
| npm | 下载和安装 JavaScript 包 | Python 的 pip |
| 全局包 | 安装成系统命令 | 安装后能直接输入 `claude` 或 `codex` |

Claude Code 官方 npm 安装命令是：

```
npm install -g @anthropic-ai/claude-code
```

Codex CLI 官方 npm 安装命令是：

```
npm install -g @openai/codex
```

这两个命令都依赖 Node.js 和 npm。

## 操作

本章先安装 Node.js LTS，然后验证 `node` 和 `npm` 命令。

## 验证

安装完成后执行：

```
node -v
npm -v
```

预期分别输出版本号。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `node` 不存在 | Node.js 未安装或 PATH 未生效 | 重启终端；仍失败则重新安装 Node.js |
| `npm` 不存在 | npm 未随 Node.js 正确安装 | 使用官方安装器重新安装 |
| npm 安装全局包失败 | 权限、网络、代理或缓存问题 | 本章后续提供排查步骤 |

# 4.2 LTS 和 Current 的区别

## 步骤

选择 Node.js LTS 版本，而不是 Current 版本。

## 原理

Node.js 官网通常提供两类版本：

| 类型 | 含义 | 适合人群 |
| --- | --- | --- |
| LTS | Long Term Support，长期支持版 | 大多数开发者、新手、生产项目 |
| Current | 最新功能版 | 想尝鲜的新功能测试者 |

新手搭建开发环境时，稳定性比新功能更重要。Claude Code 要求 Node.js 18+；Codex CLI 通过 npm 安装也需要可用的 Node/npm 环境。选择官网当前 LTS 可以兼顾兼容性和安全更新。

## 操作

访问官方页面：

```
https://nodejs.org/en/download/
```

选择：

```
Windows Installer (.msi)
LTS
x64
```

截至本文核对时，Node.js 官网显示 LTS 为 `v24.15.0 LTS`。实际安装时，以官网当前显示的 LTS 为准。

## 验证

下载文件通常类似：

```
node-v24.x.x-x64.msi
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 官网显示的版本和本文不同 | Node.js 版本持续更新 | 选择官网当前 LTS |
| 不知道 x64 还是 Arm64 | 系统架构未确认 | 回到第 1 章查看系统类型 |
| 下载到 `.zip` | 选择了二进制压缩包 | 新手应选择 `.msi` 安装器 |

# 4.3 安装 Node.js

## 步骤

运行 Node.js 官方安装器。

## 原理

官方 `.msi` 安装器会自动安装 `node.exe`、`npm`，并把 Node.js 安装目录加入 PATH。这样 PowerShell、CMD、VS Code 终端都能找到 `node` 和 `npm`。

## 操作

1. 双击 `node-v*-x64.msi`。
2. 点击 Next。
3. 接受 License。
4. 安装位置保持默认：

```
C:\Program Files\nodejs\
```

5. 组件保持默认，确保包含：
    - Node.js runtime。
    - npm package manager。
    - Add to PATH。
6. 如果出现 “Tools for Native Modules” 选项，新手可以先不勾选。少数需要编译原生扩展的 npm 包才需要它。
7. 点击 Install。
8. 安装完成后点击 Finish。

## 验证

关闭所有终端，重新打开 PowerShell：

```
node -v
npm -v
where.exe node
where.exe npm
```

预期输出类似：

```
v24.15.0
11.x.x
C:\Program Files\nodejs\node.exe
C:\Program Files\nodejs\npm
C:\Program Files\nodejs\npm.cmd
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `node -v` 无输出 | 终端异常或安装失败 | 重新打开 PowerShell；仍失败则重新安装 |
| `node` 可用但 `npm` 不可用 | npm 组件缺失或 PATH 异常 | 重新运行安装器，选择 Repair |
| 提示需要管理员 | 安装到 Program Files 需要权限 | 使用管理员批准安装，或使用官方支持的用户级安装方案 |

# 4.4 npm 基础配置

## 步骤

查看 npm 全局安装目录，确认全局命令会安装到哪里。

## 原理

执行 `npm install -g 包名` 时，npm 会把命令安装到全局目录。如果该目录不在 PATH，即使安装成功也无法输入命令运行。

## 操作

执行：

```
npm config get prefix
npm root -g
npm bin -g
```

> **Note**
> 某些新版 npm 可能不再支持 `npm bin -g`。如果报错，可使用 `npm config get prefix` 判断全局目录。

常见全局目录：

```
C:\Users\<用户名>\AppData\Roaming\npm
```

或：

```
C:\Program Files\nodejs
```

## 验证

执行：

```
$env:Path -split ';' | Select-String npm
```

预期能看到 npm 全局命令目录。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 全局安装成功但命令不可用 | npm 全局 bin 不在 PATH | 把 `C:\Users\<用户名>\AppData\Roaming\npm` 加入用户 PATH |
| npm 安装时权限不足 | 全局目录在系统目录 | 使用用户级 prefix 或以管理员安装 Node.js |
| npm 下载很慢 | 网络到 npm registry 不稳定 | 检查网络、代理，避免随意使用不可信镜像 |

# 4.5 环境变量问题排查

## 步骤

定位 Node.js PATH 问题。

## 原理

安装器修改 PATH 后，已经打开的终端不会自动刷新环境变量。VS Code 如果在安装前已打开，也会继续使用旧环境变量。

## 操作

执行：

```
where.exe node
where.exe npm
```

如果找不到，检查 Node.js 安装目录：

```
Test-Path "C:\Program Files\nodejs\node.exe"
```

如果返回 `True`，说明 Node.js 存在但 PATH 没生效。

手动添加用户 PATH：

1. 搜索“环境变量”。
2. 打开“编辑系统环境变量”。
3. 点击“环境变量”。
4. 在“用户变量”中选择 `Path`。
5. 点击“编辑”。
6. 添加：

```
C:\Program Files\nodejs\
```

7. 如果 npm 全局目录缺失，也添加：

```
C:\Users\<你的用户名>\AppData\Roaming\npm
```

8. 点击确定。
9. 关闭所有终端和 VS Code，重新打开。

## 验证

```
node -v
npm -v
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 手动加 PATH 后仍失败 | 终端没重启 | 关闭所有终端和 VS Code |
| 路径写成 `node.exe` | PATH 需要目录，不是文件 | 改成 `C:\Program Files\nodejs\` |
| 多个 Node 版本冲突 | 旧版本残留 | 使用 `where.exe node` 查看优先命中的路径 |

---

# 第 5 章 安装 Python

Python 是 AI、数据处理、脚本自动化和很多本地开发工具的基础语言。即使 Claude Code 和 Codex CLI 本身主要通过 Node/npm 安装，实际项目开发中也经常需要 Python。

## 5.1 Python 的作用

## 步骤

理解为什么 AI 开发环境必须安装 Python。

## 原理

Python 在 AI 和工程开发中非常常见：

| 场景 | Python 用途 |
| --- | --- |
| AI 应用 | 调用 OpenAI、Anthropic、向量数据库等 API |
| 数据处理 | 读取 CSV、Excel、JSON |
| 自动化 | 批量处理文件、生成报告 |
| 后端开发 | FastAPI、Flask、Django |
| 本地工具 | 运行脚本、测试、构建辅助工具 |

很多教程、开源项目、AI 生成代码都会默认你有 Python 和 pip。

## 操作

本章安装 Python 官方稳定版，并验证 `python` 与 `pip`。

## 验证

安装完成后执行：

```
python --version
pip --version
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `python` 不是内部或外部命令 | PATH 未配置 | 重新安装并勾选 Add python.exe to PATH |
| 打开 Microsoft Store | Windows Python alias 干扰 | 关闭 App execution aliases |
| `pip` 不可用 | pip 未安装或 PATH 异常 | 使用 `python -m ensurepip --upgrade` |

# 5.2 下载 Python

## 步骤

从 Python 官方网站下载 Windows 安装器。

## 原理

Python 的 Windows 安装器有多个版本。新手应选择官方稳定版 64-bit installer。不要从第三方网站下载“绿色版”或“精简版”，否则 PATH、pip、证书等问题会更难排查。

## 操作

1. 打开官方页面：https://www.python.org/downloads/windows/
2. 选择最新稳定版。
3. 下载 Windows installer 64-bit。
4. 文件名通常类似：

```
python-3.14.x-amd64.exe
```

实际版本以官网当前 Stable Release 为准。

## 验证

确认下载来源是：

```
python.org
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 看到很多版本 | Python 保留历史版本 | 选择最新 Stable Release |
| 不知道 embeddable package 是什么 | 嵌入式发行包，不适合新手 | 选择 Windows installer |
| 下载 32-bit | 老旧兼容版本 | 现代 Windows 11 通常选 64-bit |

# 5.3 安装 Python

## 步骤

运行 Python 安装器，并勾选 PATH。

## 原理

Python 安装器底部的 `Add python.exe to PATH` 是最关键选项。没有勾选时，安装后双击 Python 可能能用，但终端输入 `python` 会失败。开发环境必须让终端能找到 Python。

## 操作

1. 双击 Python 安装器。
2. 在第一个界面底部勾选：

```
Add python.exe to PATH
```

3. 点击：

```
Install Now
```

4. 等待安装完成。
5. 如果出现 `Disable path length limit`，建议点击。它可以减少 Windows 路径长度限制带来的问题。
6. 点击 Close。

## 验证

关闭所有终端，重新打开 PowerShell：

```
python --version
pip --version
where.exe python
where.exe pip
```

预期：

```
Python 3.14.x
pip 2x.x from ...
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 忘记勾选 PATH | 安装器没有配置命令路径 | 重新运行安装器，选择 Modify，勾选 PATH |
| `python` 打开 Microsoft Store | Windows 应用执行别名 | 设置 -> 应用 -> 高级应用设置 -> 应用执行别名，关闭 Python alias |
| `pip` 版本很旧 | 安装包自带版本较旧 | 执行 `python -m pip install --upgrade pip` |

# 5.4 pip 基础用法

## 步骤

学会用 `python -m pip` 安装包。

## 原理

Windows 上可能存在多个 Python。直接执行 `pip` 有时会调用到另一个 Python 的 pip。使用 `python -m pip` 能确保 pip 属于当前 `python`。

## 操作

升级 pip：

```
python -m pip install --upgrade pip
```

查看 pip 版本：

```
python -m pip --version
```

安装测试包：

```
python -m pip install requests
```

验证：

```
python -c "import requests; print(requests.__version__)"
```

### 验证

能输出 `requests` 版本号。

### 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| pip install 失败 | 网络、证书或权限问题 | 先确认网络，再尝试升级 pip |
| Permission denied | 安装到系统目录权限不足 | 使用用户安装：`python -m pip install --user 包名` |
| `import` 失败 | 包安装到另一个 Python | 使用 `python -m pip show 包名` 检查路径 |

## 5.5 PATH 修复方法

### 步骤

手动修复 Python PATH。

### 原理

Python 通常安装在用户目录，例如：

```
C:\Users\<用户名>\AppData\Local\Programs\Python\Python314\
```

pip 命令脚本通常在：

```
C:\Users\<用户名>\AppData\Local\Programs\Python\Python314\Scripts\
```

这两个目录都应加入 PATH。

### 操作

先查找 Python：

```
where.exe python
```

如果找不到，尝试：

```
Get-ChildItem "$env:LOCALAPPDATA\Programs\Python" -Recurse -Filter python.exe
```

把找到的 Python 目录和 `Scripts` 目录加入用户 PATH：

1. 搜索“环境变量”。
2. 打开“编辑系统环境变量”。
3. 点击“环境变量”。
4. 编辑用户变量 `Path`。

5. 添加类似路径：

```
C:\Users\<用户名>\AppData\Local\Programs\Python\Python314\
C:\Users\<用户名>\AppData\Local\Programs\Python\Python314\Scripts\
```

6. 保存后重启终端。

## 验证

```python --version
pip --version
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 添加后仍无效 | 终端未重启 | 完全关闭终端和 VS Code |
| 多个 Python 版本 | PATH 顺序决定优先级 | `where.exe python` 查看命中顺序 |
| pip 指向旧版本 | Scripts 目录顺序问题 | 调整 PATH 中新版 Python 的顺序靠前 |

---

# 第 6 章 安装 Claude Code

Claude Code 是 Anthropic 提供的 AI 编程代理，可以在终端中读取项目、解释代码、修改文件、运行命令并协助完成开发任务。它适合深度代码理解、项目级重构、长上下文分析和交互式编程。

## 6.1 Claude Code 介绍

### 步骤

理解 Claude Code 是什么，以及它和普通聊天机器人的区别。

### 原理

普通聊天机器人通常只能根据你粘贴的代码回答问题。Claude Code 运行在你的终端和项目目录中，可以：

1. 读取项目文件。
2. 搜索代码。
3. 生成修改建议。
4. 写入文件。
5. 执行测试或命令。
6. 解释错误日志。

因此它更像“本地代码协作者”，而不是单纯问答工具。

官方入口：

```
https://claude.ai/code
https://docs.anthropic.com/en/docs/claude-code/getting-started
```

## 操作

安装前确认：

```
node -v
npm -v
git --version
```

Claude Code 官方要求 Node.js 18+，Windows 可使用 WSL、Git for Windows 或官方 Windows 安装方式。

## 验证

上述三个命令都能输出版本号。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| Node.js 版本低于 18 | Claude Code 不满足要求 | 安装当前 LTS Node.js |
| Git 不可用 | 未安装 Git for Windows | 回到第 3 章 |
| Windows 原生命令异常 | Shell 兼容性问题 | 尝试 Git Bash 或 WSL |

# 6.2 Claude Code 的运行原理

## 步骤

了解 Claude Code 如何与你的电脑、项目和 Anthropic 服务交互。

## 原理

Claude Code 不是完全离线模型。它本地运行 CLI 程序，但 AI 推理通常发生在 Anthropic 服务端。大致流程：

1. 你在项目目录执行 `claude`。
2. Claude Code 读取项目上下文。
3. 你提出任务。
4. CLI 将必要上下文发送给 Anthropic。
5. Claude 返回分析或修改方案。
6. CLI 在本地应用修改或请求你确认命令。

这意味着：

- 需要网络连接。
- 需要 Anthropic 账号或 API Key。
- 不要在不可信项目中随意授权执行命令。

- 涉密代码要遵守公司或学校的数据合规要求。

## 操作

建议第一次使用时在测试目录运行：

```
New-Item -ItemType Directory -Force -Path C:\Dev\playground\claude-test
Set-Location C:\Dev\playground\claude-test
```

## 验证

当前目录应为：

```
Get-Location
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 担心代码泄露 | CLI 会与云端服务交互 | 不在涉密项目中测试；先阅读官方隐私和企业政策 |
| Claude 要求确认命令 | 安全机制 | 新手应逐条阅读再确认 |
| 项目太大响应慢 | 上下文和搜索成本高 | 在具体子目录启动 Claude |

# 6.3 安装 Claude Code

## 步骤

使用官方推荐命令安装 Claude Code。

## 原理

Claude Code 官方提供 npm 安装方式，也提供原生安装脚本。因为本教程已经安装 Node.js，所以优先使用 npm，便于统一管理。

## 操作

在 PowerShell 执行：

```
npm install -g @anthropic-ai/claude-code
```

如果你选择官方 Windows PowerShell 安装脚本，可参考官方文档中的命令：

```
irm https://claude.ai/install.ps1 | iex
```

> ⚠ **Warning**
> 运行远程安装脚本前必须确认来源是官方域名 `claude.ai` 或 Anthropic 官方文档。新手优先使用 npm 安装方式，更容易排查。

## 验证

```
claude --version
where.exe claude
```

然后在测试目录运行：

```
claude
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `claude` command not found | npm 全局目录不在 PATH | 检查 `npm config get prefix`，把 npm 全局目录加入 PATH |
| npm 安装权限错误 | 全局目录权限不足 | 使用用户级 npm prefix，或重新安装 Node.js |
| 安装后缺少二进制 | npm optional dependencies 被禁用或安装不完整 | 运行 `npm config get optional`，必要时重新安装 |

# 6.4 登录 Claude Code

## 步骤

首次运行 `claude`，根据提示登录。

## 原理

Claude Code 需要认证才能调用 Claude。常见认证方式：

| 方式 | 说明 | 适合人群 |
| --- | --- | --- |
| Claude.ai 订阅登录 | 使用浏览器登录 Claude 账号 | Pro/Max/Team/Enterprise 用户 |
| Anthropic Console API Key | 使用 API 计费 | 开发者、脚本、自动化 |
| 企业 SSO | 组织统一登录 | 公司或学校团队 |

需要注意：官方说明中，若设置了 `ANTHROPIC_API_KEY`，Claude Code 可能优先使用 API Key，而不是订阅额度。新手如果想使用 Claude 订阅，暂时不要设置 `ANTHROPIC_API_KEY`。

## 操作

在项目目录执行：

```
claude
```

按终端提示：

1. 选择登录方式。
2. 如果打开浏览器，登录 Claude 或 Anthropic 账号。
3. 授权 CLI。
4. 回到终端。

登录后可在 Claude Code 内运行：

```
/status
```

## 验证

能进入 Claude Code 交互界面，并看到当前认证状态。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 浏览器打不开 | 默认浏览器配置或网络问题 | 手动复制终端中的登录链接到浏览器 |
| 登录后终端无反应 | 回调失败或网络阻断 | 重新执行 `claude`，按官方提示重试 |
| 明明有订阅却走 API 计费 | 设置了 `ANTHROPIC_API_KEY` | 临时移除变量：`Remove-Item Env:ANTHROPIC_API_KEY` |

# 6.5 API 模式与订阅模式区别

## 步骤

区分 Claude Code 使用订阅登录和 API Key 的不同。

## 原理

订阅模式通常绑定 Claude.ai 账号权益；API 模式通常按 API 用量计费。两者适合不同场景。

| 对比项 | 订阅模式 | API 模式 |
| --- | --- | --- |
| 入口 | Claude.ai 登录 | Anthropic Console API Key |
| 计费 | 按订阅套餐 | 按 API 使用量 |
| 适合 | 个人日常交互式开发 | 自动化脚本、服务端调用 |
| 环境变量 | 通常不需要 `ANTHROPIC_API_KEY` | 需要 `ANTHROPIC_API_KEY` |
| 风险 | 达到套餐限制 | 可能产生额外 API 费用 |

## 操作

查看是否设置 API Key：

```
echo $env:ANTHROPIC_API_KEY
```

如果要临时设置 API Key：

```
$env:ANTHROPIC_API_KEY="your_api_key_here"
```

如果要临时删除：

```
Remove-Item Env:ANTHROPIC_API_KEY
```

## 验证

进入 Claude Code 后运行：

```
/status
```

确认当前认证方式。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| API Key 设置后仍未生效 | 终端会话或配置优先级问题 | 关闭终端重开，运行 `/status` |
| 不想用 API 计费 | 环境变量优先 | 删除 `ANTHROPIC_API_KEY` |
| API Key 泄露 | 把 Key 写进项目文件或截图 | 立即在 Console 删除旧 Key，创建新 Key |

# 6.6 网络与环境变量问题

## 步骤

定位 Claude Code 登录失败、网络错误和环境变量问题。

## 原理

Claude Code 需要访问 Anthropic 服务。如果所在地区、网络、代理、证书或公司防火墙阻止连接，就会登录失败或请求失败。

## 操作

基础检查：

```
claude --version
node -v
npm -v
git --version
```

检查环境变量：

```
echo $env:ANTHROPIC_API_KEY
echo $env:HTTP_PROXY
echo $env:HTTPS_PROXY
```

重新安装或升级：

```
npm install -g @anthropic-ai/claude-code@latest
```

运行官方诊断：

```
claude doctor
```

## 验证

`claude doctor` 不再报告关键错误，`claude` 能进入交互界面。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 登录失败 | 浏览器回调、网络或账号问题 | 复制登录链接手动打开；检查账号地区支持 |
| network error | 网络无法访问服务 | 检查代理、防火墙、DNS |
| `claude --version` 不可用 | 安装失败或 PATH 问题 | 检查 npm 全局目录 |
| API 认证失败 | `ANTHROPIC_API_KEY` 错误或过期 | 在 Anthropic Console 重新创建 Key |

---

# 第 7 章 安装 OpenAI Codex CLI

OpenAI Codex CLI 是 OpenAI 提供的本地命令行 AI 编程代理。它可以在终端中读取、修改、运行和解释代码，适合代码生成、调试、重构、测试修复和项目理解。

# 7.1 Codex CLI 介绍

## 步骤

理解 Codex CLI 是什么。

## 原理

Codex CLI 运行在你的本地终端中。它不是 VS Code 插件，也不是网页聊天，而是一个可以进入项目目录并协助你操作代码库的命令行工具。官方帮助文档说明它可以通过 npm 一条命令安装，并提供不同审批模式，让用户决定 AI 能否自动编辑文件或执行命令。

官方入口：

```
https://github.com/openai/codex
https://help.openai.com/en/articles/11096431-openai-codex-cli-getting-started
```

## 操作

安装前确认 Node.js 和 npm：

```
node -v
npm -v
```

## 验证

两个命令都输出版本号。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `npm` 不存在 | Node.js 未正确安装 | 回到第 4 章 |
| 不知道 Codex 与 ChatGPT 区别 | Codex CLI 面向本地代码库操作 | ChatGPT 更偏对话和网页工作流 |
| 担心自动改文件 | Codex 有审批模式 | 新手使用默认 Suggest 模式 |

# 7.2 Codex 与 Claude Code 的区别

## 步骤

了解两个工具的定位，避免重复安装后不知道怎么选。

## 原理

Claude Code 和 Codex CLI 都是 AI 编程代理，但模型、生态、认证方式、交互习惯不同。它们不是互斥关系，可以在同一台电脑安装。

| 对比项 | Claude Code | OpenAI Codex CLI |
| --- | --- | --- |
| 提供方 | Anthropic | OpenAI |
| 常见入口 | `claude` | `codex` |
| 安装方式 | npm 或官方安装脚本 | npm 或官方发布包 |
| 认证方式 | Claude 账号、企业账号、Anthropic API Key | OpenAI API Key 或官方支持的登录方式 |
| 适合场景 | 长上下文理解、项目对话、代码解释 | 终端代理、代码修改、测试修复、OpenAI 生态 |
| 共同点 | 都需要网络和账号 | 都应在 Git 仓库中使用 |

## 操作

建议新手：

1. 两者都安装。
2. 每次只让一个工具修改项目。
3. 修改前执行 `git status`。
4. 修改后审查差异。

## 验证

后续能分别运行：

```
claude --version
codex --version
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 两个工具都想改同一文件 | 可能产生冲突 | 一次只用一个工具完成一个任务 |
| 不知道选谁 | 任务不同 | 深度解释可先问 Claude；终端执行和修复可试 Codex |
| 输出不一致 | 模型和上下文不同 | 以测试、Git diff 和官方文档为准 |

# 7.3 安装 Codex CLI

## 步骤

使用 npm 安装 OpenAI Codex CLI。

## 原理

OpenAI 官方帮助文档和 npm 包页面都提供 npm 全局安装方式。`-g` 表示安装为全局命令，安装后可以在任意项目目录输入 `codex`。

## 操作

在 PowerShell 执行：

```
npm install -g @openai/codex
```

安装完成后查看版本：

```
codex --version
```

查看命令位置：

```
where.exe codex
```

## 验证

预期看到版本号，例如：

```
codex 0.x.x
```

实际版本以 npm 当前发布版本为准。本文核对时 npm 包页面显示 `@openai/codex` 仍在持续更新。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `codex` 不是内部或外部命令 | npm 全局目录不在 PATH | 添加 `C:\Users\<用户名>\AppData\Roaming\npm` 到 PATH |
| npm 安装失败 | 网络或 registry 访问问题 | 检查网络和代理 |
| 权限不足 | npm 全局目录需要权限 | 使用用户级 npm prefix 或修复 Node.js 安装 |

# 7.4 配置 OpenAI API Key

## 步骤

为 Codex CLI 配置 OpenAI API Key。

## 原理

API Key 是你访问 OpenAI API 的凭证。OpenAI 官方 API 文档要求使用 Bearer Token 认证，并提醒不要泄露 Key，不要把 Key 暴露在浏览器端代码或公开仓库中。很多 OpenAI 工具会自动读取环境变量：

```
OPENAI_API_KEY
```

## 操作

创建 API Key：

1. 打开：https://platform.openai.com/
2. 登录 OpenAI 账号。
3. 进入 API Keys 页面。
4. 创建新的 Secret Key。
5. 复制后保存到安全位置。

PowerShell 临时设置：

```
$env:OPENAI_API_KEY="your_openai_api_key_here"
```

验证是否设置：

```
echo $env:OPENAI_API_KEY
```

CMD 临时设置：

```
set OPENAI_API_KEY=your_openai_api_key_here
```

永久设置用户环境变量：

```
[Environment]::SetEnvironmentVariable("OPENAI_API_KEY", "your_openai_api_key_here",
"User")
```

设置后关闭所有终端和 VS Code，再重新打开。

## 验证

```
echo $env:OPENAI_API_KEY
codex --version
```

然后在测试目录执行：

```
codex
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| API Key invalid | Key 写错、过期、复制不完整 | 重新创建 Key，复制完整 |
| 终端看不到环境变量 | 永久变量需新终端读取 | 关闭终端和 VS Code 后重开 |
| 误把 Key 写进代码 | 安全风险 | 立即撤销旧 Key，使用环境变量 |

> ⚠ **Warning**
> 不要把真实 API Key 写入 Markdown、截图、GitHub、`.py` 文件、`.env` 示例文件或聊天窗口。公开泄露后必须立即删除并重新生成。

# 7.5 API 连接失败排查

## 步骤

排查 Codex CLI 无法连接 OpenAI 的问题。

## 原理

连接失败通常不是单一原因，可能来自 API Key、账号额度、网络、代理、DNS、防火墙、系统时间、CLI 版本等。

## 操作

基础检查：

```
codex --version
node -v
npm -v
echo $env:OPENAI_API_KEY
```

升级 Codex：

```
npm install -g @openai/codex@latest
```

检查代理变量：

```
echo $env:HTTP_PROXY
echo $env:HTTPS_PROXY
```

如果代理错误，临时删除：

```
Remove-Item Env:HTTP_PROXY -ErrorAction SilentlyContinue
Remove-Item Env:HTTPS_PROXY -ErrorAction SilentlyContinue
```

## 验证

再次运行：

```
codex
```

能进入交互界面并正常响应。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `API Key invalid` | Key 错误或无权限 | 重新创建 Key，确认账号项目和额度 |
| `network error` | 网络无法访问 OpenAI | 检查网络、防火墙、代理 |
| OpenAI 认证失败 | Key 未加载或登录状态异常 | 重新设置 `OPENAI_API_KEY` |
| npm 权限问题 | 全局安装目录不可写 | 修复 npm prefix 或使用管理员安装 Node.js |

---

# 第 8 章 配置 VS Code AI 开发环境

VS Code 的 AI 开发能力主要来自插件。插件不是越多越好。新手最容易犯的错误是一次安装很多 AI 插件，导致补全、聊天、右键菜单、终端代理互相冲突。

## 8.1 插件安装方法

## 步骤

学会从 VS Code Marketplace 安装插件。

## 原理

VS Code 插件扩展编辑器能力。AI 插件通常会读取当前文件、项目目录、终端输出或 Git diff，因此要选择可信插件，并理解它们的权限。

## 操作

1. 打开 VS Code。
2. 按：

```
Ctrl + Shift + X
```

3. 搜索插件名。
4. 确认发布者。
5. 点击 Install。
6. 安装后按提示登录或配置 API Key。

## 验证

插件列表中显示 `Installed`，左侧活动栏或命令面板出现对应入口。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 插件安装失败 | Marketplace 网络问题 | 检查网络，稍后重试 |
| 插件要求登录 | AI 服务需要账号 | 按插件官方说明登录 |
| 插件读取项目权限弹窗 | 安全提示 | 只在可信项目中授权 |

# 8.2 推荐 AI 插件总览

## 步骤

了解每个插件适合什么场景。

## 原理

不同插件的交互模型不同。有些偏“自动补全”，有些偏“聊天解释”，有些偏“代理式修改项目”。

| 插件 | 适合场景 | 新手建议 |
| --- | --- | --- |
| Claude Code | 在 VS Code 中配合 Claude Code 使用，适合项目级 AI 编程 | 如果你主要用 Claude，可安装 |
| GitHub Copilot | 行内补全、函数补全、聊天问答 | 推荐安装，适合日常补全 |
| Continue | 可配置多模型，适合自定义 OpenAI/Anthropic/本地模型 | 适合愿意配置模型的新手进阶 |
| Cline | 代理式修改项目、执行命令、自动化任务 | 谨慎使用，先在测试项目练习 |
| Gemini Code Assist | 使用 Google Gemini 生态 | 如果你有 Google 账号和相关需求可用 |
| Roo Code | Cline 类代理体验，适合多模式开发 | 与 Cline 二选一更稳 |
| Markdown 插件 | 写技术文档、预览 Markdown、检查格式 | 推荐安装基础 Markdown 工具 |

## 操作

新手推荐组合：

| 阶段 | 推荐组合 | 原因 |
| --- | --- | --- |
| 第 1 周 | GitHub Copilot + Markdown All in One | 少冲突，先熟悉补全和文档 |
| 第 2 周 | 加 Claude Code | 学习项目级对话 |

| 阶段 | 推荐组合 | 原因 |
| --- | --- | --- |
| 第 3 周 | Continue 或 Cline 二选一 | 开始尝试代理式工作流 |
| 进阶 | 根据模型账号选择 Gemini/Roo Code | 避免重复功能堆叠 |

## 验证

只打开一个测试项目，确认：

1. 行内补全正常。
2. AI 聊天正常。
3. 终端不被多个插件同时接管。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 多个 AI 同时补全 | 插件功能重叠 | 关闭其中一个插件的 inline completion |
| VS Code 卡顿 | 插件过多 | 禁用暂不用插件 |
| 代码被自动改乱 | 代理权限过高 | 只在 Git 仓库使用，改前提交或暂存 |

# 8.3 安装 GitHub Copilot

## 步骤

安装并登录 GitHub Copilot。

## 原理

Copilot 擅长行内补全和代码聊天。它不一定替代 Claude Code 或 Codex CLI，但能在你手写代码时持续补全，非常适合初学者提升输入效率。

## 操作

1. 在扩展市场搜索：

```
GitHub Copilot
```

2. 安装 GitHub 官方插件。
3. 安装后点击 Sign in。
4. 使用浏览器登录 GitHub。
5. 授权 VS Code。

## 验证

新建 `hello.py`：

```
def greet(name):
```

等待灰色补全提示出现，按 `Tab` 接受。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 没有补全 | 未登录或无订阅/额度 | 检查 Copilot 状态 |
| 补全太频繁 | 行内补全开启 | 在设置中调整 Copilot 自动建议 |
| 和其他插件冲突 | 多个补全源 | 禁用其他 AI 插件的 inline 功能 |

# 8.4 安装 Continue、Cline、Roo Code、Gemini Code Assist

## 步骤

按需安装代理类和多模型类插件。

## 原理

这类插件通常能力更强，也更容易误操作。它们可能读取多个文件、写入代码、执行终端命令。新手应该先在 `C:\Dev\playground` 中测试，不要一上来放到重要项目里全自动运行。

## 操作

在扩展市场分别搜索：

```
Continue
Cline
Roo Code
Gemini Code Assist
```

安装原则：

1. Continue：如果你想自己配置 OpenAI、Anthropic 或本地模型。
2. Cline：如果你想体验代理式自动开发。
3. Roo Code：如果你想尝试多模式代理工作流。
4. Gemini Code Assist：如果你使用 Google Gemini 生态。

## 验证

每次只启用一个代理类插件，在测试目录要求它完成小任务，例如：

```
请创建一个 Python 脚本，读取用户输入并打印问候语。
```

确认它修改的文件可读、可运行。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 插件要求 API Key | 需要模型服务凭证 | 按第 10 章配置 |
| 自动执行命令太多 | 代理权限设置过宽 | 关闭自动执行，改为每步确认 |
| 生成文件位置混乱 | 当前工作区未打开正确目录 | 用 VS Code 打开项目根目录 |

# 8.5 Markdown 插件配置

## 步骤

安装适合写技术文档的 Markdown 插件。

## 原理

AI 开发不仅写代码，也写 README、接口文档、部署手册、实验记录。Markdown 是最常见的轻量文档格式。

## 操作

推荐插件：

| 插件 | 用途 |
| --- | --- |
| Markdown All in One | 目录、快捷键、列表、预览增强 |
| markdownlint | 检查 Markdown 格式 |
| Markdown Preview Enhanced | 更强预览能力，可选 |

安装后打开 `.md` 文件，按：

```
Ctrl + Shift + V
```

预览 Markdown。

## 验证

创建 `README.md`：

```
# My Project

## Usage

```powershell
python app.py
```

```

能正常预览标题和代码块。

### 排错

| 现象 | 原因 | 解决方法 |
|---|---|---|
| 代码块预览异常 | Markdown 围栏未闭合 | 检查三个反引号是否成对 |
| lint 提示很多 | 规则严格 | 新手可先学习提示，不必一次全改 |
| 预览快捷键无效 | 快捷键冲突 | 命令面板搜索 Markdown Preview |

## 8.6 避免插件冲突和性能优化

### 步骤

建立“少而稳”的插件策略。

### 原理

每个 AI 插件都可能：

1. 监听你输入的代码。
2. 扫描项目文件。
3. 调用网络 API。
4. 在后台运行语言服务。
5. 修改编辑器建议列表。

插件越多，冲突和卡顿概率越高。

### 操作

禁用插件：

1. 打开扩展面板。
2. 找到插件。
3. 点击齿轮。
4. 选择 Disable。
5. 可选择 Disable (Workspace)，只在当前项目禁用。

性能建议：

```
```

```
| 建议 | 原因 |
|---|---|
| 同时只启用一个代理类插件 | 避免多个工具争抢项目控制权 |
| 大项目关闭不必要插件 | 减少后台扫描 |
| 为不同项目配置 Workspace 禁用 | 不影响全局设置 |
| 定期查看 Running Extensions | 找出耗时插件 |

打开运行中扩展：

```text
Ctrl + Shift + P
Developer: Show Running Extensions
```
```

## 验证

VS Code 启动速度正常，输入代码不卡顿，AI 补全来源清晰。

## 排错

| 现象 | 原因 | 解决方法 |
|---|---|---|
| CPU 占用高 | 插件后台索引 | 禁用可疑插件，重启 VS Code |
| 补全重复 | 多个 AI 插件提供建议 | 关闭其中一个 inline completion |
| 终端命令被插件打断 | 代理插件接管任务 | 暂停或禁用代理插件 |

---

# 第 9 章 配置终端环境

终端是开发者与系统交互的主要入口。Windows 11 上常见终端包括 PowerShell、CMD、Git Bash 和 Windows Terminal。新手需要知道它们的区别，并统一默认工作流。

## 9.1 PowerShell、CMD、Git Bash、Windows Terminal 的区别

### 步骤

区分“Shell”和“终端应用”。

### 原理

Windows Terminal 是一个终端应用，像一个外壳窗口；PowerShell、CMD、Git Bash 是 Shell，负责解释命令。可以把 Windows Terminal 理解成“容器”，里面可以打开不同 Shell。

| 名称 | 类型 | 推荐用途 |
|---|---|---|
| PowerShell | Shell | Windows 新手默认开发终端 |
| CMD | Shell | 老旧命令、部分安装器说明 |
| Git Bash | Shell | Linux/Bash 风格命令、Git 相关操作 |

| 名称 | 类型 | 推荐用途 |
| --- | --- | --- |
| Windows Terminal | 终端应用 | 管理多个 Shell 标签页 |

## 操作

分别打开：

1. 开始菜单搜索 `Windows Terminal`。
2. 新建 PowerShell 标签页。
3. 新建 Command Prompt 标签页。
4. 安装 Git 后可新建 Git Bash 标签页。

## 验证

在不同 Shell 中执行：

PowerShell：

```
$PSVersionTable.PSVersion
```

CMD：

```
ver
```

Git Bash：

```
git --version
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| Windows Terminal 找不到 Git Bash | Git 安装后未被识别 | 重启 Windows Terminal |
| CMD 中 PowerShell 命令失败 | Shell 语法不同 | 按代码块标注选择终端 |
| Git Bash 路径看不懂 | 使用 `/c/Dev` 风格 | 记住它对应 `C:\Dev` |

# 9.2 安装或更新 Windows Terminal

## 步骤

确认 Windows Terminal 可用。

## 原理

Windows 11 通常已经内置 Windows Terminal。如果没有，可通过 Microsoft Store 或 Microsoft 官方 GitHub 发布页安装。Microsoft 官方文档说明，Store 版本可自动更新；GitHub 版本适合无法访问 Store 的环境。

官方文档：

```
https://learn.microsoft.com/windows/terminal/install
```

## 操作

方法一：Microsoft Store。

1. 打开 Microsoft Store。
2. 搜索 Windows Terminal。
3. 点击安装或更新。

方法二：winget。

```
winget install --id Microsoft.WindowsTerminal -e
```

## 验证

开始菜单能打开 Windows Terminal。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| Store 无法访问 | 公司或地区限制 | 使用官方 GitHub Releases 或 winget |
| winget 不可用 | 系统组件缺失 | 使用 Store 更新 App Installer |
| 打开后默认不是 PowerShell | 默认配置不同 | 到 Settings -> Startup 修改默认 Profile |

# 9.3 把 VS Code 默认终端改成 PowerShell

## 步骤

确保 VS Code 内部终端默认是 PowerShell。

## 原理

教程中大部分 Windows 命令使用 PowerShell。如果 VS Code 默认打开 CMD 或 Git Bash，复制命令可能报错。因此先统一默认终端。

## 操作

在 VS Code 中：

1. 按 `Ctrl + Shift + P`。
2. 输入：

```
Terminal: Select Default Profile
```

3. 选择 PowerShell。
4. 关闭旧终端。
5. 新建终端。

或编辑设置：

```
{
  "terminal.integrated.defaultProfile.windows": "PowerShell"
}
```

## 验证

VS Code 终端执行：

```
$PSVersionTable.PSVersion
```

能输出版本号。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 仍打开 CMD | 旧终端未关闭 | 删除终端标签，重新新建 |
| PowerShell 不在列表 | 系统配置异常 | 安装或更新 Windows PowerShell/PowerShell |
| VS Code 终端环境变量旧 | VS Code 启动太早 | 完全退出 VS Code 后重新打开 |

# 9.4 美化终端

## 步骤

配置更清晰的终端字体、主题和提示符。

## 原理

终端美化不是为了花哨，而是为了更清楚地看到路径、Git 分支、命令状态和错误信息。新手应先追求清晰，不要安装太复杂的主题框架。

## 操作

Windows Terminal 设置：

1. 打开 Windows Terminal。
2. 点击下拉箭头。
3. 点击 Settings。
4. 选择 PowerShell Profile。
5. 设置字体为：

```
Cascadia Mono
```

6. 字号建议 12-14。
7. 配色方案可选择 One Half Dark 或 Campbell。

VS Code 终端设置：

```
{
  "terminal.integrated.fontFamily": "Cascadia Mono, Consolas, monospace",
  "terminal.integrated.fontSize": 13,
  "terminal.integrated.cursorBlinking": true
}
```

## 验证

终端文字清晰，路径和错误信息易读。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 字体乱码 | 字体不支持字符 | 换回 Cascadia Mono 或 Consolas |
| 图标显示方块 | 缺少 Nerd Font | 新手先不用图标主题 |
| 主题太暗看不清 | 对比度不足 | 选择默认浅色或 One Half Dark |

# 9.5 提高终端效率

## 步骤

掌握常用终端快捷操作。

## 原理

AI 编程仍然离不开命令行。熟悉少量高频命令能显著减少卡顿。

## 操作

PowerShell 常用命令：

| 任务 | 命令 |
| --- | --- |
| 查看当前位置 | `Get-Location` |
| 进入目录 | `Set-Location C:\Dev\projects` |
| 列出文件 | `Get-ChildItem` |
| 新建文件夹 | `New-Item -ItemType Directory -Path demo` |
| 删除临时文件 | `Remove-Item file.txt` |
| 清屏 | `Clear-Host` |
| 查看命令位置 | `where.exe node` |
| 查看环境变量 | `echo $env:OPENAI_API_KEY` |

快捷键：

| 快捷键 | 作用 |
| --- | --- |
| `Ctrl + C` | 中断当前命令 |
| `Ctrl + L` | 清屏，部分终端可用 |
| `↑` / `↓` | 查找历史命令 |
| `Tab` | 自动补全路径 |
| `Ctrl + Shift + P` | VS Code 命令面板 |
| `Ctrl + ` ` | 打开或关闭 VS Code 终端 |

## 验证

尝试：

```
Set-Location C:\Dev
Get-ChildItem
```

能看到开发目录。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 路径包含空格时报错 | 未加引号 | 使用 `Set-Location "C:\My Projects"` |
| 删除命令误删 | 对命令不熟 | 删除前先 `Get-ChildItem` 确认 |
| 命令复制后换行错误 | 代码块被错误复制 | 一次复制一条命令 |

---

# 第 10 章 API Key 配置

API Key 是 AI 开发环境中最重要、也最容易被新手误用的配置之一。本章讲清楚 API 是什么、为什么需要 Key、如何在 Windows 中保存 Key、如何避免泄露，以及如何分别配置 OpenAI 和 Anthropic。

## 10.1 API 是什么

### 步骤

理解 API 和 API Key。

### 原理

API 可以理解为软件之间通信的接口。你写的程序、Claude Code、Codex CLI 或 VS Code 插件，需要通过网络请求调用 OpenAI 或 Anthropic 的模型服务。服务提供方需要知道“是谁在调用”，这就需要 API Key。

API Key 类似一把钥匙：

| 项目 | 含义 |
| --- | --- |
| API | 程序调用服务的入口 |
| API Key | 身份凭证 |
| Token/Usage | 实际使用量 |
| Billing | 费用和额度 |

### 操作

记住两条原则：

1. 程序通过环境变量读取 Key。
2. 不把 Key 写进代码仓库。

## 验证

你应该能解释：

```
OPENAI_API_KEY 用于 OpenAI。
ANTHROPIC_API_KEY 用于 Anthropic。
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 不知道为什么要 Key | AI 服务需要认证和计费 | 先创建平台账号 |
| 把 Key 当密码保存到代码 | 安全意识不足 | 使用环境变量 |
| 多个工具读不到 Key | 环境变量名称不一致 | 按工具官方文档命名 |

# 10.2 如何避免 API Key 泄露

## 步骤

建立 API Key 安全习惯。

## 原理

API Key 泄露后，别人可能用你的额度调用 API，产生费用或造成账号风险。公开 GitHub 仓库、截图、日志、教学文档都是常见泄露渠道。

## 操作

安全规则：

| 规则 | 说明 |
| --- | --- |
| 不写入代码 | 不在 `.py`、`.js` 中硬编码 Key |
| 不提交 `.env` | `.env` 应加入 `.gitignore` |
| 不截图 | 终端回显 Key 时不要截图 |
| 不发给 AI | 不把真实 Key 粘贴到聊天窗口 |
| 定期轮换 | 怀疑泄露立即删除并重建 |

创建 `.gitignore`：

```
New-Item -ItemType File -Force -Path .gitignore
Add-Content .gitignore ".env"
Add-Content .gitignore "*.key"
```

## 验证

```
git status
```

确认 `.env` 不会被 Git 跟踪。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 已经提交 Key | Git 历史中仍有泄露 | 立即撤销 Key；必要时清理 Git 历史 |
| 插件要求填写 Key | 插件需要调用模型 | 优先使用插件的 Secret 配置或环境变量 |
| 忘记 Key 内容 | 平台通常只显示一次 | 重新创建新的 Key |

# 10.3 配置 OpenAI API Key

## 步骤

创建并配置 `OPENAI_API_KEY`。

## 原理

OpenAI API 使用 API Key 认证，通常通过 HTTP Bearer Token 发送。官方文档建议从环境变量安全读取 Key。

## 操作

创建 Key：

1. 打开 https://platform.openai.com/
2. 登录。
3. 进入 API Keys。
4. 创建新的 Secret Key。
5. 复制并安全保存。

PowerShell 临时配置：

```
$env:OPENAI_API_KEY="your_openai_api_key_here"
```

PowerShell 永久配置：

```
[Environment]::SetEnvironmentVariable("OPENAI_API_KEY", "your_openai_api_key_here",
"User")
```

CMD 临时配置：

```
set OPENAI_API_KEY=your_openai_api_key_here
```

CMD 永久配置：

```
setx OPENAI_API_KEY "your_openai_api_key_here"
```

图形界面永久配置：

1. 搜索“环境变量”。
2. 打开“编辑系统环境变量”。
3. 点击“环境变量”。
4. 在“用户变量”点击“新建”。
5. 变量名：

```
OPENAI_API_KEY
```

6. 变量值填你的 Key。
7. 点击确定。
8. 重启终端和 VS Code。

## 验证

PowerShell：

```
echo $env:OPENAI_API_KEY
```

CMD：

```
echo %OPENAI_API_KEY%
```

Python 验证读取：

```
python -c "import os; print('OK' if os.getenv('OPENAI_API_KEY') else 'MISSING')"
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 输出为空 | 没设置或终端未重启 | 重新设置并重启终端 |
| 输出旧 Key | 环境变量有多个来源 | 检查用户变量和系统变量 |
| `setx` 后当前窗口不可见 | `setx` 只影响新窗口 | 关闭并重新打开终端 |

# 10.4 配置 Anthropic API Key

## 步骤

创建并配置 `ANTHROPIC_API_KEY`。

## 原理

Anthropic API 使用 `ANTHROPIC_API_KEY`。Claude Code 官方说明：如果设置了 `ANTHROPIC_API_KEY`，Claude Code 可能优先使用 API Key，而不是 Claude.ai 订阅登录。因此新手要明确自己想用“订阅模式”还是“API 模式”。

## 操作

创建 Key：

1. 打开 https://console.anthropic.com/
2. 登录 Anthropic Console。
3. 进入 API Keys。
4. 创建 Key。
5. 复制后安全保存。

PowerShell 临时配置：

```
$env:ANTHROPIC_API_KEY="your_anthropic_api_key_here"
```

PowerShell 永久配置：

```
[Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY",
"your_anthropic_api_key_here", "User")
```

CMD 临时配置：

```
set ANTHROPIC_API_KEY=your_anthropic_api_key_here
```

CMD 永久配置：

```
setx ANTHROPIC_API_KEY "your_anthropic_api_key_here"
```

如果想临时使用 Claude 订阅，不使用 API Key：

```
Remove-Item Env:ANTHROPIC_API_KEY -ErrorAction SilentlyContinue
```

永久删除请到“环境变量”图形界面删除对应变量。

## 验证

```
echo $env:ANTHROPIC_API_KEY
```

进入 Claude Code 后：

```
/status
```

确认当前认证方式。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 有订阅却产生 API 费用 | 设置了 `ANTHROPIC_API_KEY` | 删除环境变量并重新登录 |
| API Key 无效 | Key 复制错误或已删除 | 重新创建 Key |
| VS Code 插件读不到 | 插件进程未刷新 | 重启 VS Code |

# 10.5 环境变量保存方式对比

## 步骤

选择合适的环境变量保存方式。

## 原理

临时变量只在当前终端有效，永久变量会保存到 Windows 用户环境中。新手调试时用临时变量，稳定后再写入永久变量。

| 方式 | PowerShell | CMD | 生命周期 | 推荐场景 |
| --- | --- | --- | --- | --- |
| 临时变量 | `$env:OPENAI_API_KEY="..."` | `set OPENAI_API_KEY=...` | 当前窗口 | 临时测试 |
| 永久变量 | `[Environment]::SetEnvironmentVariable(...)` | `setx` | 新窗口长期有效 | 日常开发 |
| 图形界面 | 环境变量窗口 | 环境变量窗口 | 长期有效 | 新手最直观 |
| `.env` 文件 | 需程序读取 | 需程序读取 | 项目级 | 应用开发 |

## 操作

查看当前进程变量：

```
echo $env:OPENAI_API_KEY
echo $env:ANTHROPIC_API_KEY
```

查看用户永久变量：

```
[Environment]::GetEnvironmentVariable("OPENAI_API_KEY", "User")
[Environment]::GetEnvironmentVariable("ANTHROPIC_API_KEY", "User")
```

删除用户永久变量：

```
[Environment]::SetEnvironmentVariable("OPENAI_API_KEY", $null, "User")
[Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY", $null, "User")
```

## 验证

重启终端后再次 `echo`，确认变量是否存在。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 当前窗口和新窗口结果不同 | 临时变量覆盖了永久变量 | 关闭当前窗口重开 |
| 系统变量和用户变量冲突 | 同名变量多处设置 | 优先保留用户变量，删除不需要的系统变量 |
| Key 中有特殊字符 | 命令行解析问题 | 使用引号包裹 Key |

---

# 第 11 章 第一个 AI 项目实战

本章从零创建一个 Python 小项目：一个命令行待办事项工具。你会完成项目目录创建、VS Code 打开、Git 初始化、Claude/Codex 辅助生成代码、运行项目、调试项目和提交版本。

## 11.1 创建项目文件夹

## 步骤

在规范目录下创建项目。

## 原理

每个项目都应该有独立文件夹。不要把多个项目文件混在一个目录里。AI 工具读取项目时，会以当前目录作为上下文，目录越清晰，AI 越不容易误读。

## 操作

打开 PowerShell：

```
New-Item -ItemType Directory -Force -Path C:\Dev\projects\ai-todo-python
Set-Location C:\Dev\projects\ai-todo-python
```

创建基础文件：

```
New-Item -ItemType File -Force -Path README.md
New-Item -ItemType File -Force -Path app.py
New-Item -ItemType File -Force -Path requirements.txt
New-Item -ItemType File -Force -Path .gitignore
```

写入 `.gitignore`：

```
Add-Content .gitignore ".venv/"
Add-Content .gitignore "__pycache__/"
Add-Content .gitignore ".env"
Add-Content .gitignore "*.pyc"
```

## 验证

```
Get-ChildItem
```

预期看到：

```
.gitignore
README.md
app.py
requirements.txt
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 创建目录失败 | 没有权限 | 改用用户目录下的 `Dev` |
| 文件没有扩展名 | 文件扩展名未显示 | 回到第 1 章开启扩展名 |
| `.gitignore` 看不到 | 隐藏文件未显示 | 文件资源管理器显示隐藏项目 |

# 11.2 用 VS Code 打开项目

## 步骤

用 VS Code 打开项目根目录。

## 原理

AI 插件和 Git 面板都依赖工作区根目录。如果只打开单个文件，VS Code 不知道项目结构，Claude/Codex 插件也无法完整理解上下文。

## 操作

在项目目录执行：

```
code .
```

打开后确认 VS Code 左侧资源管理器显示项目文件。

## 验证

VS Code 窗口标题或资源管理器显示：

```
ai-todo-python
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| `code .` 不可用 | VS Code PATH 未配置 | 回到第 2 章修复 |
| 只打开了文件不是文件夹 | 使用了文件打开 | 在 VS Code 中选择 File -> Open Folder |
| 终端路径不对 | VS Code 未在项目根目录打开 | 终端执行 `Get-Location` 确认 |

# 11.3 初始化 Git

## 步骤

把项目变成 Git 仓库。

## 原理

在 AI 修改代码前先初始化 Git，可以随时查看 AI 改了什么。Codex CLI 官方也会提醒在版本控制目录中使用更安全。

## 操作

```
git init
git status
git add .
git commit -m "Initial project structure"
```

## 验证

```
git log --oneline
```

预期看到一次提交：

```
xxxxxxx Initial project structure
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| Git 要求设置 user.name | 未配置 Git 身份 | 回到第 3 章配置 |
| commit 打不开编辑器 | 默认编辑器异常 | 使用 `git commit -m "message"` |
| `.venv` 被提交 | `.gitignore` 不完整 | 添加 `.venv/` 后重新检查 |

# 11.4 创建 Python 虚拟环境

## 步骤

为项目创建独立 Python 环境。

## 原理

虚拟环境可以让每个项目拥有自己的依赖，避免全局 Python 包互相污染。例如一个项目需要新版库，另一个项目需要旧版库，虚拟环境可以隔离。

## 操作

```
python -m venv .venv
```

激活虚拟环境：

```
.\.venv\Scripts\Activate.ps1
```

如果提示脚本禁止运行，回到第 1 章设置执行策略：

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

升级 pip：

```
python -m pip install --upgrade pip
```

## 验证

终端前面出现：

```
(.venv)
```

查看 Python 路径：

```
where.exe python
```

预期优先显示项目 `.venv` 下的 Python。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 无法激活 | PowerShell 执行策略限制 | 设置 `RemoteSigned` |
| 激活后 Python 仍是全局 | 终端没有进入项目目录 | `Set-Location C:\Dev\projects\ai-todo-python` |
| `.venv` 出现在 Git 变更 | `.gitignore` 未生效 | 确认 `.gitignore` 包含 `.venv/` |

# 11.5 编写第一个程序

## 步骤

创建一个简单待办事项 CLI。

## 原理

这个项目不依赖第三方包，适合验证 Python、VS Code、终端和 Git 工作流。它会把任务保存到本地 `todo.json`，包含添加、列表、完成任务功能。

## 操作

在 `app.py` 写入：

```
import json
from pathlib import Path

DATA_FILE = Path("todo.json")


def load_tasks():
```

```python
    if not DATA_FILE.exists():
        return []
    return json.loads(DATA_FILE.read_text(encoding="utf-8"))


def save_tasks(tasks):
    DATA_FILE.write_text(json.dumps(tasks, ensure_ascii=False, indent=2), encoding="utf-8")


def list_tasks(tasks):
    if not tasks:
        print("No tasks yet.")
        return
    for index, task in enumerate(tasks, start=1):
        mark = "x" if task["done"] else " "
        print(f"{index}. [{mark}] {task['title']}")


def add_task(tasks, title):
    tasks.append({"title": title, "done": False})
    save_tasks(tasks)
    print(f"Added: {title}")


def complete_task(tasks, number):
    index = number - 1
    if index < 0 or index >= len(tasks):
        print("Task number does not exist.")
        return
    tasks[index]["done"] = True
    save_tasks(tasks)
    print(f"Completed: {tasks[index]['title']}")


def main():
    tasks = load_tasks()
    while True:
        print("\nAI Todo")
        print("1. List tasks")
        print("2. Add task")
        print("3. Complete task")
        print("4. Exit")
        choice = input("Choose: ").strip()

        if choice == "1":
            list_tasks(tasks)
        elif choice == "2":
            title = input("Task title: ").strip()
            if title:
                add_task(tasks, title)
        elif choice == "3":
            number_text = input("Task number: ").strip()
```

```
            if number_text.isdigit():
                complete_task(tasks, int(number_text))
            else:
                print("Please enter a number.")
        elif choice == "4":
            break
        else:
            print("Unknown choice.")


if __name__ == "__main__":
    main()
```

## 验证

运行：

```
python app.py
```

尝试：

1. 输入 `2` 添加任务。
2. 输入 `1` 查看任务。
3. 输入 `3` 完成任务。
4. 输入 `4` 退出。

预期生成：

```
todo.json
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 中文乱码 | 终端编码或字体问题 | Windows Terminal 使用 UTF-8 友好字体 |
| `json.decoder.JSONDecodeError` | `todo.json` 内容损坏 | 删除 `todo.json` 重新运行 |
| 输入后没保存 | 程序未写入文件 | 检查当前目录是否是项目根目录 |

# 11.6 用 Claude/Codex 辅助改进代码

## 步骤

让 AI 在项目中提出改进建议。

## 原理

AI 工具适合做明确任务。不要只说“帮我优化”，而要给出边界和验收标准。例如：增加删除任务功能、增加命令行参数、补充 README、添加测试。

## 操作

使用 Claude Code：

```
claude
```

输入任务：

```
请阅读当前 Python 项目，为 app.py 增加“删除任务”功能。要求：
1. 菜单中新增 Delete task。
2. 用户输入任务编号后删除。
3. 编号不存在时给出友好提示。
4. 不引入第三方依赖。
5. 修改后说明你改了哪些地方。
```

使用 Codex CLI：

```
codex
```

输入类似任务。

## 验证

AI 修改后先查看差异：

```
git diff
```

运行程序：

```
python app.py
```

确认删除功能可用后提交：

```
git add .
git commit -m "Add delete task feature"
```

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| AI 改坏了代码 | 任务描述不清或自动修改过多 | 用 `git diff` 查看，必要时手动修正 |
| AI 执行危险命令 | 授权前未阅读 | 新手不要开启全自动执行 |

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| 两个 AI 同时修改 | 工作流冲突 | 一次只使用一个工具 |

# 11.7 调试方法

## 步骤

学会最基础的调试方式。

## 原理

调试不是只看错误。你需要知道程序执行到哪里、变量是什么、输入输出是否符合预期。VS Code 支持断点调试，Python 也可以用 `print` 快速检查。

## 操作

方法一：print 调试。

在 `load_tasks` 中临时加入：

```
print("Loading tasks from", DATA_FILE)
```

运行：

```
python app.py
```

方法二：VS Code 断点。

1. 打开 `app.py`。
2. 点击行号左侧添加红点。
3. 按 `F5`。
4. 选择 Python Debugger。
5. 观察 VARIABLES 面板。

## 验证

程序在断点处暂停，可以看到 `tasks` 变量。

## 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| F5 不工作 | Python 扩展未安装 | 安装 Microsoft Python 扩展 |
| 断点不停 | 运行的不是当前文件 | 选择 Python File 调试配置 |
| 虚拟环境未识别 | VS Code Python 解释器未选 | `Ctrl+Shift+P` -> Python: Select Interpreter |

# 11.8 README 和运行说明

## 步骤

写项目说明文档。

## 原理

一个项目即使很小，也应该说明如何安装、运行、使用和调试。README 是人类和 AI 都会优先阅读的项目入口。

## 操作

在 `README.md` 写入：

```
# AI Todo Python

一个适合新手练习的 Python 命令行待办事项项目。

## 环境要求

- Windows 11
- Python 3.14 或官网当前稳定版

## 运行方法

```powershell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python app.py
```

# 功能

- 查看任务
- 添加任务
- 完成任务
- 本地 JSON 保存

# 调试

在 VS Code 中打开 `app.py`，添加断点后按 `F5`。

### 验证

VS Code 中按：

```text
Ctrl + Shift + V
```
```

能正常预览 README。

### 排错

| 现象 | 原因 | 解决方法 |
| --- | --- | --- |
| Markdown 代码块乱了 | 反引号未闭合 | 确认每个代码块有开始和结束 |
| 命令不能运行 | 虚拟环境未激活 | 先执行 Activate |
| README 与代码不一致 | 后续修改忘更新 | 每次功能变化同步更新 README |

---

# 第 12 章 常见报错大全

本章按工具分类列出新手最常遇到的问题。排错顺序建议：先看命令是否存在，再看 PATH，再看网络，再看账号/API Key，再看工具版本。

## 12.1 Node 相关报错

### 报错：`npm 不是内部或外部命令`

| 项目 | 内容 |
| --- | --- |
| 原因 | Node.js 未安装、npm 未安装、PATH 未生效 |
| 排查思路 | 先执行 `node -v`，再执行 `where.exe npm` |
| 解决方法 | 重新安装 Node.js LTS，并确保 Add to PATH；重启终端 |

命令：

```
node -v
npm -v
where.exe npm
```

### 报错：`node 不存在`

| 项目 | 内容 |
| --- | --- |
| 原因 | Node.js 没安装或安装目录不在 PATH |
| 排查思路 | 检查 `C:\Program Files\nodejs\node.exe` 是否存在 |
| 解决方法 | 安装 Node.js LTS；手动添加 `C:\Program Files\nodejs\` 到 PATH |

命令：

```
Test-Path "C:\Program Files\nodejs\node.exe"
```

### 报错：`npm install` 失败

| 项目 | 内容 |
| --- | --- |
| 原因 | 网络问题、权限问题、npm registry 访问失败、缓存损坏 |
| 排查思路 | 查看错误中是否有 `EACCES`、`ETIMEDOUT`、`ENOTFOUND` |
| 解决方法 | 检查网络；重试；清理缓存；确认全局目录权限 |

命令：

```
npm cache verify
npm config get registry
npm config get prefix
```

## 12.2 Python 相关报错

### 报错：`python 不是内部或外部命令`

| 项目 | 内容 |
| --- | --- |
| 原因 | Python 未安装、PATH 未勾选、Windows alias 干扰 |
| 排查思路 | 执行 `where.exe python`，检查是否打开 Microsoft Store |
| 解决方法 | 重新安装 Python 并勾选 Add python.exe to PATH；关闭应用执行别名 |

命令：

```
python --version
where.exe python
```

### 报错：`pip 无法使用`

| 项目 | 内容 |
| --- | --- |
| 原因 | pip 未安装、Scripts 目录不在 PATH、pip 属于另一个 Python |
| 排查思路 | 使用 `python -m pip --version` |
| 解决方法 | 使用 `python -m ensurepip --upgrade`，并添加 Scripts 到 PATH |

命令：

```
python -m pip --version
python -m ensurepip --upgrade
```

### 报错：`pip install` 失败

| 项目 | 内容 |
| --- | --- |
| 原因 | 网络、证书、权限、Python 版本不兼容 |
| 排查思路 | 看错误关键词：`timeout`、`SSL`、`Permission denied`、`No matching distribution` |
| 解决方法 | 升级 pip；检查网络；必要时使用 `--user`；确认包支持当前 Python |

命令：

```
python -m pip install --upgrade pip
python -m pip install --user requests
```

## 12.3 Git 相关报错

### 报错：`git 不是内部或外部命令`

| 项目 | 内容 |
| --- | --- |
| 原因 | Git 未安装或 PATH 未配置 |
| 排查思路 | 执行 `where.exe git` |
| 解决方法 | 安装 Git for Windows，并选择 command line PATH 选项 |

命令：

```
git --version
where.exe git
```

### 报错：`git clone` 失败

| 项目 | 内容 |
| --- | --- |
| 原因 | 网络、仓库地址错误、权限不足、SSH Key 未配置 |
| 排查思路 | 区分 HTTPS 和 SSH；检查错误是 authentication 还是 network |
| 解决方法 | 公共仓库用 HTTPS；私有仓库登录 GitHub；SSH 方式需配置 SSH Key |

命令：

```
git clone https://github.com/openai/codex.git
```

如果私有仓库失败，先确认：

```
git remote -v
git config --global --list
```

## 12.4 Claude 相关报错

### 报错：`claude command not found`

| 项目 | 内容 |
| --- | --- |
| 原因 | Claude Code 未安装、npm 全局目录不在 PATH、终端未重启 |
| 排查思路 | 执行 `npm list -g --depth=0` 和 `where.exe claude` |
| 解决方法 | 重新安装 `@anthropic-ai/claude-code`；修复 npm PATH |

命令：

```
npm install -g @anthropic-ai/claude-code
claude --version
where.exe claude
```

### 报错：Claude 登录失败

| 项目 | 内容 |
| --- | --- |
| 原因 | 浏览器回调失败、账号问题、网络问题、地区支持问题 |
| 排查思路 | 看是否能打开登录链接；检查是否有 `ANTHROPIC_API_KEY` 干扰 |
| 解决方法 | 复制链接手动打开；运行 `claude doctor`；确认账号可用 |

命令：

```
claude doctor
echo $env:ANTHROPIC_API_KEY
```

### 报错：Claude 网络错误

| 项目 | 内容 |
| --- | --- |
| 原因 | 网络无法访问 Anthropic、代理错误、防火墙阻止 |
| 排查思路 | 检查浏览器是否能访问 Claude；检查代理变量 |
| 解决方法 | 修复网络；移除错误代理；使用受支持网络环境 |

命令：

```
echo $env:HTTP_PROXY
echo $env:HTTPS_PROXY
```

## 12.5 Codex 相关报错

### 报错：`API Key invalid`

| 项目 | 内容 |
| --- | --- |
| 原因 | OpenAI API Key 错误、过期、复制不完整、项目权限不对 |
| 排查思路 | 查看 `OPENAI_API_KEY` 是否存在；重新生成 Key |
| 解决方法 | 在 OpenAI 平台创建新 Key；重启终端；不要复制多余空格 |

命令：

```
echo $env:OPENAI_API_KEY
```

### 报错：`network error`

| 项目 | 内容 |
| --- | --- |
| 原因 | 网络连接失败、DNS、代理、防火墙、服务临时不可用 |
| 排查思路 | 判断浏览器能否访问 OpenAI 平台；检查代理变量 |
| 解决方法 | 修复网络；关闭错误代理；稍后重试；升级 Codex CLI |

命令：

```
codex --version
npm install -g @openai/codex@latest
```

### 报错：OpenAI 认证失败

| 项目 | 内容 |
| --- | --- |
| 原因 | Key 未配置、Key 与账号项目不匹配、额度不足 |
| 排查思路 | 检查平台 Billing、API Keys、项目权限 |
| 解决方法 | 配置正确 Key；确认账号有 API 权限和额度 |

命令：

```
[Environment]::GetEnvironmentVariable("OPENAI_API_KEY", "User")
```

# 12.6 通用排错流程

## 步骤

遇到任何命令行工具不可用时，按固定顺序排查。

## 原理

大多数问题都可以归类为：没安装、PATH 不对、版本不对、网络不通、认证失败、权限不足。

## 操作

通用检查清单：

```
git --version
node -v
npm -v
python --version
pip --version
claude --version
codex --version
```

检查命令路径：

```
where.exe git
where.exe node
where.exe npm
where.exe python
where.exe pip
where.exe claude
where.exe codex
```

检查 API Key：

```
echo $env:OPENAI_API_KEY
echo $env:ANTHROPIC_API_KEY
```

检查当前目录是否是 Git 仓库：

```
git status
```

## 验证

所有核心命令都能输出版本号，项目目录中 `git status` 能正常显示状态。

## 排错

| 问题类型 | 判断方式 | 处理方法 |
| --- | --- | --- |
| 未安装 | `where.exe` 找不到 | 回到对应安装章节 |
| PATH 问题 | 软件存在但命令找不到 | 添加安装目录到 PATH，重启终端 |
| 网络问题 | install/login/API 超时 | 检查网络、防火墙、代理 |
| 权限问题 | `EACCES`、拒绝访问 | 使用用户目录或管理员安装 |
| API 问题 | invalid/auth failed | 重新创建 Key，检查环境变量 |
| 项目问题 | AI 不理解文件 | 用 VS Code 打开项目根目录，并初始化 Git |

# 12.7 最终环境验收

## 步骤

执行最终验收命令，确认环境部署完成。

## 原理

安装成功不等于环境可用。最终验收要覆盖编辑器、终端、Git、Node、Python、Claude、Codex、API Key。

## 操作

打开 PowerShell：

```
code --version
git --version
node -v
npm -v
python --version
pip --version
claude --version
codex --version
echo $env:OPENAI_API_KEY
echo $env:ANTHROPIC_API_KEY
```

创建验收项目：

```
New-Item -ItemType Directory -Force -Path C:\Dev\playground\final-check
Set-Location C:\Dev\playground\final-check
git init
code .
```

## 验证

你应达到以下状态：

| 能力 | 验收标准 |
| --- | --- |
| VS Code | `code .` 能打开项目 |
| Git | `git status` 可用 |
| Node/npm | `node -v`、`npm -v` 可用 |
| Python/pip | `python --version`、`pip --version` 可用 |
| Claude Code | `claude --version` 可用，能登录 |
| Codex CLI | `codex --version` 可用，能进入交互 |
| API Key | 需要 API 模式时环境变量可读取 |
| 项目开发 | 能创建、运行、调试 Python 项目 |

## 排错

如果任意一项失败，不要重装所有工具。只回到对应章节按“五段式”排查：

1. 命令是否存在。
2. 版本是否符合要求。
3. PATH 是否正确。
4. 网络是否正常。
5. 认证是否有效。

---

# 附录 A：Pandoc 转 Word 方法

如果后续需要把本文转换为 Word 文档，可以安装 Pandoc 后执行：

```
pandoc Claude_Codex_Deployment_Guide.md -o "Claude Code + Codex 本地开发环境完整部署教程.docx"
```

如果需要更好的 Word 样式，可以使用参考模板：

```
pandoc Claude_Codex_Deployment_Guide.md --reference-doc=reference.docx -o "Claude Code + Codex 本地开发环境完整部署教程.docx"
```

---

# 附录 B：长期维护建议

1. 每月检查一次 Node.js LTS、Python、Git、VS Code 更新。

2. Claude Code 和 Codex CLI 使用 `npm install -g 包名@latest` 升级。

3. API Key 定期轮换，旧 Key 不再使用就删除。

4. 所有正式项目必须使用 Git。

5. AI 修改代码前先提交一个稳定版本。

6. 不要在涉密项目中随意使用云端 AI 工具。

7. 新插件先在 `C:\Dev\playground` 测试，再用于正式项目。